CONSULTAS CRUZADAS ENTRE DICIONÁRIOS

Agora que você domina os fundamentos, vamos **usar um dicionário para consultar outro e criar um terceiro**.

O PROBLEMA FUNDAMENTAL DAS CONSULTAS CRUZADAS

EXEMPLO 1 O Cenário Clássico

Imagine que você trabalha em uma empresa e tem duas tabelas:

TABELA 1: Relação de funcionários e seus departamentos
`João → Vendas`
`Maria → TI`
`Pedro → Vendas`
`Ana → RH`

TABELA 2: Relação de departamentos e seus gerentes
`Vendas → Carlos`
`TI → Patricia`
`RH → Marcelo`

PERGUNTA: Quem é o gerente de cada funcionário?
Para responder, você precisa:

1. Acessar o funcionário
2. Descobrir em qual departamento ele trabalha (TABELA 1)
3. Usar esse departamento para descobrir o gerente (TABELA 2)

ESTE É O PADRÃO FUNDAMENTAL: Um dicionário fornece a chave para consultar outro dicionário.

Visualização do Problema	

Regra Geral da Consulta Cruzada

Forma geral	Dicionário
`A → B`	Primeiro dicionário
`B → C`	Segundo dicionário
Resultado: `A → C`	Dicionário final

Sempre pergunte:

1. **Qual é a chave inicial?**
2. **Qual valor ela retorna?**
3. **Esse valor vira chave de qual dicionário?**
4. **Qual é o valor final?**

💡 Os 5 Passos Universais das Consultas Cruzadas com Dicionários

Ideia central: Quando um exercício tem **dois ou mais dicionários**, normalmente precisamos usar o **valor de um como chave de outro**.

Etapas do Método

Passo	Pergunta que o programador faz	Ação no código	Exemplo
1	Qual dicionário contém as **chaves principais**?	Escolher o dicionário de partida	`funcionario_depto`
2	Qual informação precisamos descobrir primeiro?	Obter o **valor associado**	`depto = funcionario_depto["João"]`
3	Esse valor serve de chave para qual outro dicionário?	Usar o valor para nova consulta	`gerente = depto_gerente[depto]`
4	Qual será a estrutura do resultado final?	Criar novo par no dicionário	`funcionario_gerente["João"] = gerente`
5	Repetir o processo para todos os elementos	Automatizar depois com loop	mesmo processo para Maria, Pedro, Ana

---

Código

```
NOVO DICIONÁRIO (funcionario_gerente):
- João → ? (precisamos descobrir)
- Maria → ? (precisamos descobrir)
- etc.
```

➢ Criando os Dicionários de Origem

```python
# DICIONÁRIO 1: Mapeia cada funcionário para seu departamento
# CHAVE: nome do funcionário (string)
# VALOR: nome do departamento (string)
funcionario_depto = {
   "João": "Vendas",
   "Maria": "TI",
   "Pedro": "Vendas",
   "Ana": "RH",
   "Carlos": "TI",    # Carlos também é funcionário
   "Luciana": "Marketing"  # Luciana está em um departamento que pode não ter gerente
}

# DICIONÁRIO 2: Mapeia cada departamento para seu gerente
# CHAVE: nome do departamento (string)
# VALOR: nome do gerente (string)
depto_gerente = {
   "Vendas": "Carlos",    # Carlos é gerente de Vendas
   "TI": "Patricia",     # Patricia é gerente de TI
   "RH": "Marcelo",      # Marcelo é gerente de RH
   # "Marketing" não tem gerente cadastrado!
}
```

➢ Criando o Dicionário Resultante

```python
# Dicionário de destino (inicia vazio)
funcionario_gerente = {}

print("INÍCIO DO PROCESSO AUTOMÁTICO:")
print("-" * 40)

# PARA CADA funcionário no primeiro dicionário
for (funcionario, departamento) in funcionario_depto.items():
   print(f"\nProcessando: {funcionario}")
   print(f"  → Trabalha no departamento: {departamento}")

   # ACESSA no segundo dicionário o gerente deste departamento
   gerente = depto_gerente.get(departamento)
   print(f"  → Gerente encontrado: {gerente}")

   # GUARDA no novo dicionário
   funcionario_gerente[funcionario] = gerente
   print(f"  → Adicionado ao novo dicionário: {funcionario} → {gerente}")

print("\n" + "=" * 40)
print("RESULTADO FINAL:")
print("Dicionário funcionario_gerente:")
for (func, ger) in funcionario_gerente.items():
   print(f"  {func} → {ger}")
```

SAÍDA DO PROCESSO:

```
INÍCIO DO PROCESSO AUTOMÁTICO:
----------------------------------------

Processando: João
  → Trabalha no departamento: Vendas
  → Gerente encontrado: Carlos
  → Adicionado ao novo dicionário: João → Carlos

Processando: Maria
  → Trabalha no departamento: TI
  → Gerente encontrado: Patricia
  → Adicionado ao novo dicionário: Maria → Patricia

Processando: Pedro
  → Trabalha no departamento: Vendas
  → Gerente encontrado: Carlos
  → Adicionado ao novo dicionário: Pedro → Carlos

Processando: Ana
  → Trabalha no departamento: RH
  → Gerente encontrado: Marcelo
  → Adicionado ao novo dicionário: Ana → Marcelo

Processando: Carlos
  → Trabalha no departamento: TI
  → Gerente encontrado: Patricia
  → Adicionado ao novo dicionário: Carlos → Patricia

Processando: Luciana
  → Trabalha no departamento: Marketing
  → Gerente encontrado: None
  → Adicionado ao novo dicionário: Luciana → None

========================================
RESULTADO FINAL:
Dicionário funcionario_gerente:
  João → Carlos
  Maria → Patricia
  Pedro → Carlos
  Ana → Marcelo
  Carlos → Patricia
  Luciana → None
```

Revisando:

1. **Padrão de Consulta Cruzada:** Para cada elemento do primeiro dicionário, usamos seu VALOR como CHAVE para consultar o segundo dicionário.
2. **Uso do `.get()` é FUNDAMENTAL:** No caso da Luciana, o departamento "Marketing" não existia no segundo dicionário. Se tivéssemos usado `depto_gerente[departamento]`, o programa teria parado com erro! O `.get()` salvou o programa, retornando `None`.
3. **Estrutura do Novo Dicionário:** O novo dicionário tem:
   - CHAVES: as mesmas chaves do primeiro dicionário (funcionários)
   - VALORES: os valores encontrados no segundo dicionário (gerentes)

TRATANDO CASOS ESPECIAIS: O Problema dos Valores Ausentes

No exemplo anterior, Luciana ficou com gerente `None`. Isso pode não ser desejável. Muitas vezes queremos um valor padrão.

```python
# Dicionários de origem (mesmos do exemplo anterior)
funcionario_depto = {
  "João": "Vendas",
  "Maria": "TI",
  "Pedro": "Vendas",
  "Ana": "RH",
  "Carlos": "TI",
  "Luciana": "Marketing"
}
```

```python
depto_gerente = {
  "Vendas": "Carlos",
  "TI": "Patricia",
  "RH": "Marcelo"
}
```

```python
# SOLUÇÃO 1: Usar valor padrão no .get()
func_gerente_padrao = {}

for (func, dep) in funcionario_depto.items():
  # Se não encontrar gerente, coloca "Gerente não designado"
  gerente = depto_gerente.get(departamento, "Gerente não designado")
  func_gerente_padrao[funcionario] = gerente

print("=== COM VALOR PADRÃO ===")
for (func, ger) in func_gerente_padrao.items():
  print(f"  {func} → {ger}")
```

```python
# SOLUÇÃO 2: Só incluir se encontrou (filtrar)
func_gerente_filtrado = {}

for (func, dep) in funcionario_depto.items():
  gerente = depto_gerente.get(departamento)

  # Só adiciona se gerente não for None
  if gerente is not None:
    func_gerente_filtrado[funcionario] = gerente
  else:
    print(f"  ⚠ {func} não incluído (departamento {dep} sem gerente)")

print("\n=== APENAS QUEM TEM GERENTE ===")
for func, ger in func_gerente_filtrado.items():
  print(f"  {func} → {ger}")
```

EXEMPLO 2 - NOTAS DE ALUNOS POR TURMA

O Problema

Temos:

- **Dicionário 1:** Aluno → Turma
- **Dicionário 2:** Turma → Lista de disciplinas
- **Dicionário 3:** (Aluno, Disciplina) → Nota

Queremos criar: Aluno → Média das notas (considerando apenas as disciplinas da sua turma)

Aqui, a lógica é em três camadas. Primeiro, você descobre a turma. Depois, descobre as disciplinas da turma. Por fim, consulta a nota do aluno em cada disciplina usando uma chave composta.

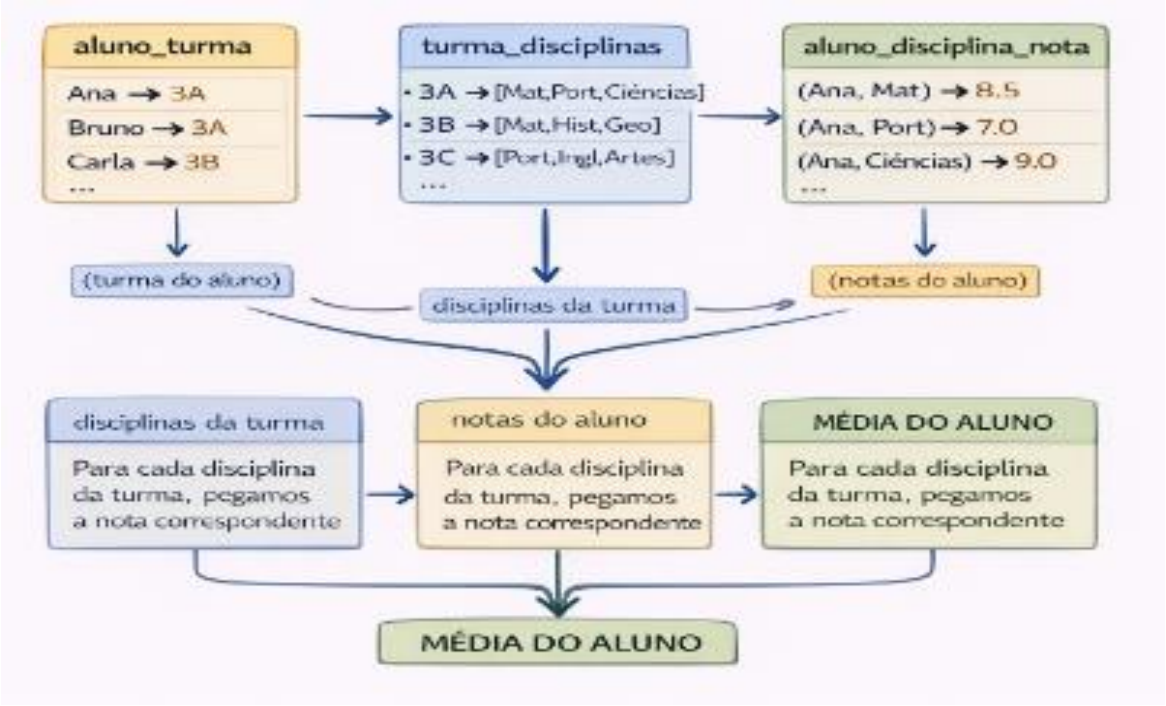
➢ Criando os Dicionários de Origem

```
# DICIONÁRIO 1: Aluno -> Turma
aluno_turma = {
  "Ana": "3A",
  "Bruno": "3A",
  "Carla": "3B",
  "Daniel": "3B",
  "Elena": "3A",
  "Fernando": "3C"
}

# DICIONÁRIO 2: Turma -> Disciplinas (lista)
turma_disciplinas = {
  "3A": ["Matemática", "Português", "Ciências"],
  "3B": ["Matemática", "História", "Geografia"],
  "3C": ["Português", "Inglês", "Artes"]
}

# DICIONÁRIO 3: (Aluno, Disciplina) -> Nota
# Como a chave é composta, vamos usar uma tupla (aluno, disciplina)
aluno_disciplina_nota = {
  ("Ana", "Matemática"): 8.5,
  ("Ana", "Português"): 7.0,
  ("Ana", "Ciências"): 9.0,
  ("Bruno", "Matemática"): 6.5,
  ("Bruno", "Português"): 8.0,
  ("Bruno", "Ciências"): 7.5,
  ("Carla", "Matemática"): 9.5,
  ("Carla", "História"): 8.0,
  ("Carla", "Geografia"): 8.5,
  ("Daniel", "Matemática"): 7.0,
  ("Daniel", "História"): 6.5,
  ("Daniel", "Geografia"): 7.0,
  ("Elena", "Matemática"): 9.0,
  ("Elena", "Português"): 8.5,
  ("Elena", "Ciências"): 9.5,
  # Fernando não tem nenhuma nota cadastrada!
}
```

Visualização:



Solução

```python
# DICIONÁRIO 1: Aluno -> Turma
# Mapeia cada aluno para sua respectiva turma
aluno_turma = {
    "Ana": "3A",
    "Bruno": "3A",
    "Carla": "3B",
    "Daniel": "3B",
    "Elena": "3A",
    "Fernando": "3C"
}

# DICIONÁRIO 2: Turma -> Disciplinas
# Mapeia cada turma para a lista de disciplinas que ela oferece
turma_disciplinas = {
    "3A": ["Matemática", "Português", "Ciências"],
    "3B": ["Matemática", "História", "Geografia"],
    "3C": ["Português", "Inglês", "Artes"]
}

# DICIONÁRIO 3: (Aluno, Disciplina) -> Nota
# Usa tuplas (aluno, disciplina) como chave para armazenar as notas
aluno_disciplina_nota = {
    ("Ana", "Matemática"): 8.5,
    ("Ana", "Português"): 7.0,
    ("Ana", "Ciências"): 9.0,
    ("Bruno", "Matemática"): 6.5,
    ("Bruno", "Português"): 8.0,
    ("Bruno", "Ciências"): 7.5,
    ("Carla", "Matemática"): 9.5,
    ("Carla", "História"): 8.0,
    ("Carla", "Geografia"): 8.5,
    ("Daniel", "Matemática"): 7.0,
```

```python
    ("Daniel", "História"): 6.5,
    ("Daniel", "Geografia"): 7.0,
    ("Elena", "Matemática"): 9.0,
    ("Elena", "Português"): 8.5,
    ("Elena", "Ciências"): 9.5,
    # Fernando não tem notas cadastradas - proposital para testar caso sem notas
}

# Dicionário que vai armazenar o resultado final: aluno -> média
aluno_media = {}

print("=== SISTEMA DE CÁLCULO DE MÉDIAS ===")
print("=" * 40)

# Percorre todos os alunos do dicionário aluno_turma
for aluno, turma in aluno_turma.items():
    print(f"\n▶ Aluno: {aluno} | Turma: {turma}")

    # Busca as disciplinas da turma do aluno
    # O método get() retorna None se a turma não existir
    disciplinas = turma_disciplinas.get(turma)

    # Verifica se a turma existe no dicionário de disciplinas
    if not disciplinas:
        print(f"  ⚠ Turma {turma} não encontrada")
        aluno_media[aluno] = "Turma inválida"  # Registra erro no resultado
    else:
        # Mostra as disciplinas que o aluno deveria cursar
        print(f"  📚 Disciplinas: {', '.join(disciplinas)}")

        # Variáveis para calcular a média
        soma_notas = 0      # Acumulador para somar todas as notas
        total_disciplinas = 0  # Contador de disciplinas com nota

        # Percorre cada disciplina da turma
        for disciplina in disciplinas:
            # Cria a chave composta (tupla) para buscar a nota
            chave = (aluno, disciplina)
            # Tenta encontrar a nota do aluno nesta disciplina
            nota = aluno_disciplina_nota.get(chave)

            if nota is not None:
                # Se encontrou a nota, exibe e adiciona ao cálculo
                print(f"  ✓ {disciplina}: {nota}")
                soma_notas += nota
                total_disciplinas += 1
            else:
                # Se não encontrou a nota, informa que está faltando
                print(f"  ✗ {disciplina}: sem nota")

        # Após verificar todas as disciplinas, calcula a média
        if total_disciplinas > 0:
            # Calcula a média (soma dividida pela quantidade)
            media = soma_notas / total_disciplinas
            # Armazena a média arredondada com 2 casas decimais
            aluno_media[aluno] = round(media, 2)
            print(f"  📊 Média final: {media:.2f}")
        else:
            # Caso o aluno não tenha nenhuma nota
            aluno_media[aluno] = "Sem notas"
```

```
            print(f"  📊 Média final: Sem notas")

# Após processar todos os alunos, exibe o resultado final
print("\n" + "=" * 40)
print("📋 RESUMO DAS MÉDIAS:")
print("-" * 25)
# Percorre e mostra o resultado de cada aluno
for (aluno, media) in aluno_media.items():
    print(f"{aluno:8} → {media}")
```

Revisando:

Conceitos importantes deste exemplo:

1. **Chaves Compostas:** Usamos uma tupla `(aluno, disciplina)` como chave no terceiro dicionário. Isso é muito comum quando a relação depende de dois fatores.
2. **Múltiplas Consultas Cruzadas:** Para cada aluno, precisamos:
    - Consultar o dicionário `turma_disciplinas` usando a turma do aluno
    - Para cada disciplina obtida, consultar o dicionário `aluno_disciplina_nota`
3. **Acumulação de Resultados:** Vamos acumulando as notas em uma lista para depois calcular a média.
4. **Tratamento de Múltiplos Cenários de Ausência:**
    - Turma sem disciplinas cadastradas
    - Disciplina sem nota para o aluno
    - Aluno sem nenhuma nota

EXEMPLO 3 - FOLHA DE PAGAMENTO

12.1 O Problema

Uma empresa precisa calcular o salário final de cada funcionário. Para isso, temos três fontes de informação:

1. **DICIONÁRIO 1 (funcionario_cargo):** Diz em qual cargo cada funcionário trabalha
2. **DICIONÁRIO 2 (cargo_salario_base):** Diz qual é o salário base de cada cargo
3. **DICIONÁRIO 3 (funcionario_dependentes):** Diz quantos dependentes cada funcionário tem

REGRAS DE NEGÓCIO:

- Cada dependente dá um acréscimo de R$ 150,00 ao salário
- Salário final = salário_base + (número_dependentes × 150)

➢ **Dicionários de Origem**

```
# DICIONÁRIO 1: Funcionário → Cargo
funcionario_cargo = {
```

```
    "João": "Analista",
    "Maria": "Gerente",
    "Pedro": "Analista",
    "Ana": "Diretor",
    "Carlos": "Assistente",
    "Luciana": "Gerente",
    "Roberto": "Analista",
    "Patricia": "Diretor"
}

# DICIONÁRIO 2: Cargo → Salário Base
cargo_salario_base = {
    "Analista": 3500.00,
    "Gerente": 5800.00,
    "Diretor": 8500.00,
    "Assistente": 2200.00,
}

# DICIONÁRIO 3: Funcionário → Número de Dependentes
funcionario_dependentes = {
    "João": 2,
    "Maria": 1,
    "Pedro": 0,
    "Ana": 3,
    "Carlos": 1,
    "Luciana": 2,
    # "Roberto" e "Patricia" não têm dependentes cadastrados
}

# VALOR DO BENEFÍCIO POR DEPENDENTE (constante)
BENEFICIO_POR_DEPENDENTE = 150.00
```

➢ Solução

```
# Dicionário de destino (inicia vazio)
funcionario_salario_final = {}

print("\n" + "=" * 70)
print("INICIANDO CÁLCULO DOS SALÁRIOS FINAIS")
print("=" * 70)

# PERCORRENDO o primeiro dicionário (funcionario_cargo)
for funcionario, cargo in funcionario_cargo.items():
    print(f"\n{'-' * 50}")
    print(f"👤 PROCESSANDO FUNCIONÁRIO: {funcionario}")
    print(f"{'-' * 50}")

    # ────────────────────────────────────────────────────────────
    # ETAPA 1: Acessar  o salário base do cargo
    # ────────────────────────────────────────────────────────────
    print(f"\n🔍 ETAPA 1: Buscando salário base para o cargo '{cargo}'")

    salario_base = cargo_salario_base.get(cargo)

    if salario_base is None:
        print(f"  ❌ ERRO: Cargo '{cargo}' não encontrado no dicionário de salários!")
        print(f"  → Funcionário {funcionario} não terá salário calculado.")
        continue  # Pula para o próximo funcionário
```

```
    print(f"  ✅ Salário base encontrado: R$ {salario_base:.2f}")

    # ────────────────────────────────────────────────────────────
    # ETAPA 2: Buscar número de dependentes
    # ────────────────────────────────────────────────────────────
    print(f"\n🔍 ETAPA 2: Acessando número de dependentes de {funcionario}")

    # IMPORTANTE: .get() com valor padrão 0
    # Se o funcionário não estiver no dicionário, consideramos 0 dependentes
    dependentes = funcionario_dependentes.get(funcionario, 0)

    print(f"  → Dependentes encontrados: {dependentes}")

    # ────────────────────────────────────────────────────────────
    # ETAPA 3: Calcular benefício e salário final
    # ────────────────────────────────────────────────────────────
    print(f"\n🧮 ETAPA 3: Calculando valores")

    beneficio = dependentes * BENEFICIO_POR_DEPENDENTE
    salario_final = salario_base + beneficio

    print(f"  → Benefício ({dependentes} × R$ 150,00) = R$ {beneficio:.2f}")
    print(f"  → Salário base: R$ {salario_base:.2f}")
    print(f"  → SALÁRIO FINAL: R$ {salario_final:.2f}")

    # ────────────────────────────────────────────────────────────
    # ETAPA 4: Guardar no novo dicionário
    # ────────────────────────────────────────────────────────────
    funcionario_salario_final[funcionario] = round(salario_final, 2)
    print(f"\n✅ Dados salvos no novo dicionário!")

# ────────────────────────────────────────────────────────────────
# RESULTADO FINAL
# ────────────────────────────────────────────────────────────────
print("\n" + "=" * 70)
print("RESULTADO FINAL - SALÁRIOS CALCULADOS")
print("=" * 70)

for funcionario, salario in funcionario_salario_final.items():
    cargo = funcionario_cargo[funcionario]
    dependentes = funcionario_dependentes.get(funcionario, 0)
    salario_base = cargo_salario_base[cargo]
    beneficio = dependentes * BENEFICIO_POR_DEPENDENTE

    print(f"\n👤 {funcionario} ({cargo})")
    print(f"  Base: R$ {salario_base:.2f} + {dependentes} dep. (R$ {beneficio:.2f}) = R$ {salario:.2f}")
```

MÓDULO 10: OS 4 PADRÕES DE CONSULTA CRUZADA

Padrão	Ideia central
Corrente simples	O valor do dicionário A vira chave no dicionário B, produzindo um novo mapeamento.

Dupla consulta	Usamos dois dicionários para obter duas informações que serão combinadas.
Acumulação ou Agrupamento	Vários itens são contados/agrupados sob a mesma chave de categoria.
Chave composta	A consulta depende de mais de um fator, então a chave precisa ser uma tupla.

Como decidir qual padrão usar
• Pergunte primeiro: qual será a chave do meu resultado?
• Depois pergunte: de onde virá o valor final?
• Se o valor final depende de outra estrutura, você provavelmente está em uma consulta cruzada.
• Se depende de duas informações ao mesmo tempo, considere chave composta ou dupla consulta.

Padrão 1: Cruzamento Simples

```python
"""
CARACTERÍSTICAS:
- Parte de um dicionário A (chaves que nos interessam)
- Usa o VALOR de A como CHAVE em B
- Resultado: dicionário com as mesmas chaves de A
"""

def padrao_cruzamento_simples(dic_a, dic_b):
    """
    dic_a: {chave: valor_intermediario}
    dic_b: {valor_intermediario: resultado_final}

    Retorna: {chave: resultado_final}
    """
    resultado = {}

    for chave, valor_intermediario in dic_a.items():
        # O valor de A vira chave em B
        resultado_final = dic_b.get(valor_intermediario)

        if resultado_final is not None:
            resultado[chave] = resultado_final

    return resultado

# Exemplo de uso
funcionario_cargo = {"João": "Vendas", "Maria": "TI"}
cargo_salario = {"Vendas": 3000, "TI": 4000}
```

```python
print(padrao_corrente_simples(funcionario_cargo, cargo_salario))
# Resultado: {"João": 3000, "Maria": 4000}
```

Padrão 2: Dupla Consulta

```python
"""
CARACTERÍSTICAS:
- Parte de um dicionário A
- Consulta dois dicionários diferentes (B e C)
- Combina os resultados
"""

def padrao_dupla_consulta(dic_a, dic_b, dic_c, funcao_combinar):
    """
    dic_a: {chave: valor_b}
    dic_b: {valor_b: resultado1}
    dic_c: {chave: resultado2} (pode usar a mesma chave ou outra)

    Retorna: {chave: funcao_combinar(resultado1, resultado2)}
    """
    resultado = {}

    for chave, valor_b in dic_a.items():
        resultado1 = dic_b.get(valor_b)
        resultado2 = dic_c.get(chave, 0)  # valor padrão 0

        if resultado1 is not None:
            resultado[chave] = funcao_combinar(resultado1, resultado2)

    return resultado
```

Padrão 3: Acumulação/Agrupamento (a ser detalhado posteriormente)

```python
"""
CARACTERÍSTICAS:
- Parte de um dicionário A (item → categoria)
- Usa a categoria para agrupar itens
- Resultado: {categoria: [lista_de_itens]} ou Resultado: {categoria: qt de itens}
"""
def padrao_acumulacao(dic_item_categoria):
    """
    dic_item_categoria: {item: categoria}

    Retorna: {categoria: qt itens}
    """
    resultado = {}

    for item, categoria in dic_item_categoria.items():
        # Se a categoria ainda não existe no resultado, retorna 0, senão retorna qt atual
        qtAtual= resultado.get(categoria, 0)

        # Adiciona 1 a qt atual  da categoria
        qtAtual += 1

        # inclui/atualiza a categoria no dicionário
```

```python
        resultado[categoria] = qtAtual

    return resultado

def padrao_agrupamento_forma1(dic_item_categoria):
    """
    dic_item_categoria: {item: categoria}

    Retorna: {categoria: [lista de itens]}
    """
    resultado = {}

    for item, categoria in dic_item_categoria.items():
        # Se a categoria ainda não existe no resultado, retorna lista vazia, senão lista atual
        lAtual=resultado.get(categoria, [])

        # Adiciona o item à lista da categoria
        lAtual.append(item)

        # inclui/atualiza a categoria no dicionário
            resultado[categoria]=lAtual

    return resultado

def padrao_agrupamento_forma2(dic_item_categoria):
    """
    dic_item_categoria: {item: categoria}

    Retorna: {categoria: [lista de itens]}
    """
    resultado = {}

    for item, categoria in dic_item_categoria.items():
        # Se a categoria ainda não existe no resultado, inclui lista vazia; acessa a lista atual
            if categoria not in resultado:
                    resultado[categoria]=[]
            lAtual=resultado.get(categoria)

        # Adiciona o item à lista da categoria ( como a lista é mutável, já atualizou o dicionário
        lAtual.append(item)


    return resultado

# Exemplo
produtos = {"Notebook": "Eletrônicos", "Mouse": "Eletrônicos", "Camisa": "Vestuário"}
print(padrao_acumulacao(produtos))
# Resultado: {"Eletrônicos": ["Notebook", "Mouse"], "Vestuário": ["Camisa"]}
```

Padrão 4: Chave Composta

```python
"""
CARACTERÍSTICAS:
- A chave para consulta é formada por dois ou mais valores
- Geralmente usamos uma TUPLA como chave
"""

def padrao_chave_composta(lista_de_pares, dic_valores):
```

```python
    """
    lista_de_pares: lista de tuplas (chave1, chave2)
    dic_valores: {(chave1, chave2): valor}

    Retorna: {(chave1, chave2): valor}
    """
    resultado = {}

    for par in lista_de_pares:
        # O par já é uma tupla, pode ser usado como chave
        valor = dic_valores.get(par)

        if valor is not None:
            resultado[par] = valor

    return resultado

# Exemplo
pares = [("Ana", "Matemática"), ("Ana", "Português"), ("Bruno", "Matemática")]
notas = {("Ana", "Matemática"): 8.5, ("Ana", "Português"): 7.0}

print(padrao_chave_composta(pares, notas))
# Resultado: {("Ana", "Matemática"): 8.5, ("Ana", "Português"): 7.0}
```

Resumo dos padrões:

Padrão	Quando Usar	Estrutura de Origem	Estrutura de Destino	Exemplo Típico
Cruzamento Simples	Valor de A vira chave em B	A: {X: Y} B: {Y: Z}	{X: Z}	Funcionário → Departamento → Gerente
Dupla Consulta	Precisamos de duas informações diferentes	A: {X: Y} B: {Y: Z1} C: {X: Z2}	{X: Z1 + Z2}	Salário + Dependentes
Acumulação	Agrupar itens por categoria	A: {item: cat}	{cat: [itens]}	Produtos por categoria
Chave Composta	Relação depende de dois fatores	Lista de pares D: {(a,b): valor}	{(a,b): valor}	Notas por aluno e disciplina

---

EXERCÍCIO FINAL INTEGRADOR

PROBLEMA:

Uma escola tem:

- **aluno_turma:** aluno → turma
- **turma_professor:** turma → professor
- **professor_email:** professor → email
- **aluno_nota_final:** aluno → nota

Crie um dicionário **contato_responsavel** que, para cada aluno, tenha o email do professor da sua turma, mas APENAS se o aluno tiver nota final maior ou igual a 7.0.

DICIONÁRIOS:

```python
aluno_turma = {
  "Ana": "3A",
  "Bruno": "3B",
  "Carla": "3A",
  "Daniel": "3C",
  "Elena": "3B"
}

turma_professor = {
  "3A": "Carlos",
  "3B": "Patricia",
  "3C": "Marcelo"
}

professor_email = {
  "Carlos": "carlos@escola.com",
  "Patricia": "patricia@escola.com",
  "Marcelo": "marcelo@escola.com"
}

aluno_nota_final = {
  "Ana": 8.5,
  "Bruno": 6.5,
  "Carla": 9.0,
  "Daniel": 5.0,
  "Elena": 7.5
}
```

SOLUÇÃO:

```python
contato_responsavel = {}

print("=== GERANDO CONTATOS DOS RESPONSÁVEIS ===")
print("(Apenas alunos com nota ≥ 7.0)\n")

for (aluno, turma) in aluno_turma.items():
  print(f"\n📝 Processando aluno: {aluno}")

  # PASSO 1: Verificar nota do aluno
  nota = aluno_nota_final.get(aluno)

  if nota is None:
    print(f"  ⚠ Aluno sem nota cadastrada")
  else:
    print(f"  → Nota: {nota}")

    # PASSO 2: Filtrar pela nota
    if nota < 7.0:
      print(f"  → Nota abaixo de 7.0. Não será incluído.")
    else:
      print(f"  → Nota OK (≥ 7.0)")

      # PASSO 3: Descobrir o professor da turma
      professor = turma_professor.get(turma)
```

```
        if professor is None:
          print(f"  ⚠ Turma {turma} sem professor cadastrado")
        else:
          print(f"  → Professor da turma: {professor}")

          # PASSO 4: Descobrir email do professor
          email = professor_email.get(professor)

          if email is None:
            print(f"  ⚠ Professor {professor} sem email cadastrado")
          else:
            print(f"  → Email do professor: {email}")

            # PASSO 5: Guardar no novo dicionário
            contato_responsavel[aluno] = email
            print(f"  ✅ Contato salvo: {aluno} → {email}")

print("\n" + "=" * 50)
print("CONTATOS DOS RESPONSÁVEIS (alunos com nota ≥ 7.0):")
for aluno, email in contato_responsavel.items():
  print(f"  {aluno}: {email}")
```

Exemplo de saída esperada:

```
=== GERANDO CONTATOS DOS RESPONSÁVEIS ===
(Apenas alunos com nota ≥ 7.0)

📝 Processando aluno: Ana
  → Nota: 8.5
  → Nota OK (≥ 7.0)
  → Professor da turma: Carlos
  → Email do professor: carlos@escola.com
  ✅ Contato salvo: Ana → carlos@escola.com

📝 Processando aluno: João
  → Nota: 6.5
  → Nota abaixo de 7.0. Não será incluído.

📝 Processando aluno: Maria
  ⚠ Aluno sem nota cadastrada

==================================================
CONTATOS DOS RESPONSÁVEIS (alunos com nota ≥ 7.0):
  Ana: carlos@escola.com
```

Lembre-se sempre:

1. **Escolha o dicionário de partida** com base nas chaves que você quer no resultado
2. **Use `.get()` em TODAS as consultas** para não correr risco de erro
3. **Decida o que fazer com dados ausentes**: ignorar, usar valor padrão, ou pular
4. **Documente o fluxo** para não se perder em consultas complexas

---

⚡ Diferença fundamental para listas

Estrutura	Como encontra um elemento	O que acontece internamente
Lista	Busca sequencial	Percorre elemento por elemento
Dicionário	Acesso pela chave	Não precisa procurar